Literature Review
P2P File Synchronization System

Action Learning Project

Backend (Go, C++)

Arihant Jain
Su Huizhe

Frontend and CI/CD (Java, GitHub Actions)

Nizar Soufiya
Shengkang Duan

May 29, 2026

Contents

1 Problem Context

1.1 Scenario and Pain Points

To motivate the problem, we begin with a concrete collaborative scenario commonly observed in university projects.

University students frequently work in temporary groups and need to exchange files efficiently without relying on complex infrastructure, cloud account management, or external storage devices. During the course of a project, team members collaboratively produce and modify heterogeneous files, including source code, documents, images, datasets, and presentation materials. Since collaborators often use different operating systems and development environments, cross-platform compatibility further increases the complexity of file sharing and coordination.

In practice, temporary student teams usually lack the time, resources, and technical expertise required to maintain a dedicated file management or synchronization system. As a result, files are often exchanged through ad hoc communication channels such as group chats or email attachments. This approach introduces several issues, including duplicated files, inconsistent versions, lack of synchronization, and absence of proper change tracking.

In addition, ownership and persistence of project data become problematic once the collaboration ends. Some members may wish to preserve the project files for future reference, while others may prefer to discard them. However, because the files are typically distributed informally and managed inconsistently, long-term access often depends on a single individual maintaining the archive. This creates risks of data loss, incomplete project history, and fragmented ownership.

Therefore, the students require a lightweight and decentralized solution that enables them to:

- create a shared repository without complicated server deployment or administrative setup;
- support synchronization of heterogeneous file types across multiple platforms;
- track modifications and maintain version history;
- provide shared ownership of project data among collaborators; and
- avoid dependence on a single participant for long-term storage, archival, or deletion of the project files.

1.2 Problem Definition

Based on the scenario described above, small-scale collaborative groups require a file-sharing mechanism that is lightweight, decentralized, and easy to use. In such environments, participants mainly care about whether they can quickly form a shared workspace, exchange project files reliably, preserve previous versions when changes are made, and continue working with the shared folder as if it were a normal local directory.

From this objective, the problem can be defined as the design of a peer-to-peer directory-level synchronization system with lightweight version control. More specifically, the system must solve the following technical problems:

- **Membership Management.**

  The system must allow collaborators to discover peers, establish sharing relationships, and manage participation in a shared workspace without relying on centralized account management. It should support dynamic membership, allowing peers to join, leave, or temporarily disconnect during collaboration.

- **Peer-to-Peer Synchronization**

  The system must automatically propagate updates among participating peers without requiring manual file exchange. It should support multi-peer collaboration, tolerate temporary peer disconnections, and maintain a reasonably consistent shared workspace during active collaboration.

- **Version Management.**

  The system must provide lightweight version management and rollback capabilities. Before a synchronized update overwrites an existing file, an earlier state should be preserved to allow recovery from accidental modifications, incorrect updates, or synchronization errors.

- **Heterogeneous File Support.**

  The system must support heterogeneous project artifacts, including source code, documents, images, multimedia content, datasets, and other binary files. In addition, large files should be transferred efficiently without assuming that all data can be exchanged through small messages.

- **Transparent Filesystem Integration.**

  The shared workspace should behave like a normal directory on the user's computer. Users should be able to create, edit, move, and delete files using their existing tools, while synchronization and version management are handled transparently in the background.

Therefore, the central technical problem is to design a serverless peer-to-peer directory synchronization system that provides effective membership management, automatic synchronization, lightweight history management, support for heterogeneous project files, and transparent filesystem integration while preserving the simplicity of ordinary local folder-based collaboration.

2 Overview of the Existing Work

Existing work related to collaborative file management can be approached from several perspectives. Some systems focus on providing shared repositories and file sharing through managed services, while others focus on preserving file history and supporting collaborative modifications. Another category of systems addresses directory synchronization among distributed devices, and some technologies focus on enabling direct file exchange between nearby users.

To provide a structured overview of the existing landscape, the related work discussed in this section is grouped into four categories: centralized collaboration platforms, collaborative version management systems, peer-to-peer synchronization systems, and local sharing technologies. Together, these systems address different aspects of file sharing, synchronization, version history management, and collaborative work.

This section introduces representative systems and their primary functions. A detailed comparison of their approaches and their relevance to the problem defined in this project will be discussed in the later sections.

2.1 Centralized Collaboration Platforms

Centralized collaboration platforms are systems that provide file storage, sharing, and synchronization services through a centrally managed infrastructure. Users interact with a common server or

cloud service that stores project data and coordinates access among collaborators. These platforms are widely used for collaborative work and document management.

Common functionalities provided by centralized collaboration platforms include cloud-based file storage, folder sharing, cross-device synchronization, and user permission management. These features enable multiple users to access and modify shared files through a unified platform.

Representative examples include *Google Drive*, *Dropbox*,*Microsoft OneDrive*, and *Nextcloud*. Although these systems differ in deployment models and implementation details, they share a common architecture in which a central service manages data storage, synchronization, and user access.

2.2 Collaborative Version Control Systems

Collaborative Version control systems are software tools that support collaborative work by managing changes made to shared files over time. They allow multiple contributors to work on the same project while maintaining a structured history of modifications and preserving previous versions of files.

The main functions of these version control systems include change tracking, version history management, conflict resolution, and rollback. By recording how files evolve over time, these systems help collaborators coordinate their work, review previous modifications, recover from incorrect changes, and maintain consistency within a shared project.

Representative examples include *Git*, *Apache Subversion (SVN)*, and *Perforce*. Although these systems differ in architecture and workflow, they all provide mechanisms for collaborative development and controlled management of file revisions.

2.3 Peer-to-Peer Synchronization Systems

Peer-to-peer synchronization systems are distributed applications that synchronize files directly between participating devices without relying on a central storage server. Each peer maintains its own copy of the shared data and exchanges updates with other peers in order to keep folders synchronized.

The main functions of peer-to-peer synchronization systems include decentralized data ownership, direct peer-to-peer communication, automatic peer discovery, and distributed replication of shared folders. By distributing both storage and synchronization responsibilities among participating devices, these systems enable collaborative file sharing without requiring dedicated server infrastructure.

Representative examples include *Syncthing* and *Resilio Sync*. Unlike traditional cloud-based solutions, these systems allow participating devices to communicate directly with one another and collectively maintain shared data.

2.4 Local Sharing Technologies

Local sharing technologies enable nearby devices to exchange files and other resources directly within a local environment. Unlike cloud-based collaboration platforms, these technologies typically allow users to share data without first uploading it to a remote server.

The main functions of local sharing technologies include direct device-to-device file transfer, temporary resource sharing, automatic discovery of nearby devices, and simplified local connectivity. By reducing the need for external infrastructure, these technologies provide a convenient mechanism for local data exchange.

Representative examples include *AirDrop*, and *Nearby Share*. These systems are commonly used for temporary file exchange and temporary collaboration between nearby devices.

3 Comparison of Approaches

We compare collaborative file synchronization tools by setup effort, server requirements, and capability to synchronize heterogeneous files across local meshes [1–6].

3.1 Comparison Dimensions

The approaches introduced in the previous section address different aspects of collaborative file management. To compare their suitability for the problem considered in this project, we evaluate them according to five key dimensions:

1. **Membership Management.** The ability to discover peers, support dynamic participation, and establish collaborative groups without extensive manual configuration or centralized management.
2. **Synchronization Model.** The ability to propagate changes automatically across multiple devices, support asynchronous collaboration, tolerate temporary disconnections, and maintain shared directory consistency.
3. **History Management.** The ability to preserve previous file states, provide rollback mechanisms, and manage conflicting modifications made by different collaborators.
4. **File Support.** The ability to handle heterogeneous file types, including source code, documents, images, multimedia content, datasets, and other binary files, as well as support efficient transfer and synchronization of large files.
5. **Filesystem Transparency and Usability.** The extent to which users can interact with shared files through ordinary filesystem operations without requiring specialized workflows, commands, or collaboration tools.

These dimensions capture the core requirements identified in the problem definition and provide a common basis for comparing existing approaches.

3.2 Centralized Collaboration Platforms

Representative examples of centralized collaboration platforms include *Google Drive*, *Microsoft OneDrive*, *Dropbox*, and *Nextcloud*. *Google Drive* and *OneDrive* are cloud services operated by large providers and offer tightly integrated ecosystems for document sharing and synchronization. *Dropbox* focuses primarily on cross-device file synchronization and collaborative storage, while *Nextcloud* provides a self-hosted alternative that allows organizations and users to deploy and manage their own collaboration infrastructure.

3.2.1 Membership Management

Centralized collaboration platforms generally provide strong membership management through user accounts, invitations, permission settings, and access control mechanisms. Collaborative groups can

be created and managed easily, but participation is typically governed by a central authority or service provider.

For example, *Google Drive* and *OneDrive* allow users to invite collaborators through email-based accounts and assign different access permissions.

3.2.2 Synchronization Model

These platforms provide automatic synchronization across multiple devices by maintaining a central copy of shared data. Updates are propagated through the cloud service and become available to collaborators once synchronization is completed. Such systems support multi-device collaboration, although they generally depend on connectivity to the central service.

For example, a document modified in *Google Drive* is uploaded to the cloud service and subsequently synchronized to other authorized devices connected to the same workspace.

3.2.3 History Management

Most centralized collaboration platforms provide basic version history and file recovery capabilities. Previous revisions can often be restored, but history management is usually less comprehensive than that offered by dedicated version control systems.

For example, *Google Drive* maintains previous revisions of documents and allows users to restore earlier versions when necessary.

3.2.4 File Support

Centralized collaboration platforms support a wide range of file types, including source code, documents, images, multimedia content, datasets, and other binary assets. They also support large files, although storage quotas and service limitations may affect practical usage.

For example, *Dropbox* and *Nextcloud* can store source code, documents, images, videos, and other binary assets within the same shared workspace.

3.2.5 Filesystem Transparency and Usability

These systems generally provide strong filesystem transparency. Users interact with shared files through familiar folder-based interfaces and synchronization clients, allowing collaborative workflows to be integrated into normal file management practices with minimal additional training.

For example, *Dropbox* provides a synchronized local folder that can be accessed through standard filesystem operations without requiring users to interact directly with the underlying synchronization mechanisms.

In summary, centralized collaboration platforms provide strong support for membership management, synchronization, file compatibility, and usability. Their centralized design simplifies collaboration and reduces the technical burden on users. However, these systems depend on centralized infrastructure for storage and coordination, which introduces reliance on external services and limits direct peer-to-peer collaboration.

3.3 Collaborative Version Control Systems

Representative examples of collaborative version control systems include *Git*, *Apache Subversion (SVN)*, and *Perforce*. *Git* is a distributed version control system in which each participant maintains a local copy of the repository history. *SVN* follows a centralized repository model, while *Perforce* is commonly used in large-scale collaborative projects involving both source code and binary assets.

3.3.1 Membership Management

Collaborative version control systems support collaboration through shared repositories and controlled contribution workflows. Users can join projects, obtain repository access, and contribute changes according to repository permissions.

For example, *Git* repositories can be distributed among multiple collaborators, while centralized systems such as *SVN* and *Perforce* typically manage access through a central repository server.

3.3.2 Synchronization Model

Version control systems are not primarily designed for continuous file synchronization. Instead, changes are usually exchanged through explicit operations such as commits, pushes, updates, or pulls.

For example, a modification made in *Git* becomes available to other collaborators only after the change has been committed and shared through the repository workflow. Consequently, synchronization is typically user-driven rather than automatic.

3.3.3 History Management

History management is the primary strength of collaborative version control systems. These systems maintain detailed revision histories and provide mechanisms for reviewing, restoring, and comparing previous versions of files.

For example, *Git* records changes as commits and supports branching, merging, rollback, and conflict resolution. Similar capabilities are also provided by *SVN* and *Perforce* through their respective revision management workflows.

3.3.4 File Support

Version control systems can manage a wide variety of file types, including source code, documents, images, multimedia content, datasets, and binary assets.

However, support for large files and frequently changing binary content varies significantly between systems. For example, *Git* is primarily optimized for text-based artifacts such as source code, whereas *Perforce* provides stronger support for large binary assets commonly encountered in game development and media production.

3.3.5 Filesystem Transparency and Usability

Version control systems typically require users to interact with repositories through dedicated workflows and tools. Operations such as committing, updating, merging, and resolving conflicts introduce additional complexity compared with ordinary file management.

For example, users of *Git* must explicitly manage repository state and revision history rather than relying on fully transparent background synchronization.

3.4 Peer-to-Peer Synchronization Systems

Representative examples of peer-to-peer synchronization systems include *Syncthing* and *Resilio Sync*. Both systems are designed to synchronize shared folders across multiple devices without relying on centralized cloud storage. Instead, participating devices communicate directly with one another and collectively maintain replicated copies of shared data.

3.4.1 Membership Management

In peer-to-peer synchronization systems, membership management is usually organized around devices, shared folders, and access credentials rather than centralized user accounts. A peer must be authorized before it can participate in the synchronization of a shared directory.

For example, *Syncthing* manages membership through device identifiers and folder sharing approvals. A device can join a synchronized folder only after the relevant peers approve the sharing relationship. *Resilio Sync* uses secret keys or sharing links to grant access to synchronized folders, so possession of the appropriate key determines whether a device can join the group.

3.4.2 Synchronization Model

Synchronization is the primary strength of peer-to-peer synchronization systems. File changes are automatically detected and propagated among participating devices without requiring explicit user intervention.

For example, when a file is modified within a synchronized folder in *Syncthing*, the update is automatically distributed to other connected devices. Changes may also be propagated after temporary disconnections, allowing participants to continue working while offline and synchronize once connectivity is restored.

3.4.3 History Management

Peer-to-peer synchronization systems generally provide limited support for history management. Their primary objective is to maintain consistent copies of files rather than preserve comprehensive revision histories.

For example, *Syncthing* offers file versioning mechanisms that can retain previous copies of modified files. However, these mechanisms are typically based on snapshots or archived versions rather than structured revision tracking.

As a result, rollback capabilities are usually available, but conflict resolution and long-term history management remain less sophisticated than those provided by dedicated version control systems.

3.4.4 File Support

Peer-to-peer synchronization systems support a wide variety of file types, including source code, documents, images, multimedia content, datasets, and other binary assets.

For example, both *Syncthing* and *Resilio Sync* can synchronize heterogeneous project directories containing mixed file types without requiring specialized handling for individual formats.

Large files are also supported, making these systems suitable for collaborative projects involving multimedia content or other storage-intensive resources.

3.4.5 Filesystem Transparency and Usability

Peer-to-peer synchronization systems typically integrate directly with the local filesystem through ordinary directories. Users can create, modify, move, and delete files using their preferred tools while synchronization occurs in the background.

For example, synchronized folders in *Syncthing* appear as normal directories on the local machine, allowing users to work without interacting directly with the synchronization protocol.

This approach provides strong filesystem transparency and requires relatively little workflow adaptation compared with traditional version control systems.

3.5 Local Sharing Technologies

Representative examples of local sharing technologies include *AirDrop* and *Nearby Share*. These systems are designed to enable direct file exchange between nearby devices without requiring users to upload data to a centralized cloud service.

Unlike peer-to-peer synchronization systems, local sharing technologies primarily focus on one-time file transfers rather than maintaining continuously synchronized directories. They are commonly used for temporary file exchange and short-term collaboration among nearby users.

3.5.1 Membership Management

Local sharing technologies generally provide lightweight mechanisms for discovering nearby devices and initiating file transfers. Membership is typically established on demand when a user chooses to share a file with another device.

For example, *AirDrop* automatically discovers nearby Apple devices and allows users to approve incoming transfer requests. Similarly, *Nearby Share* enables temporary sharing relationships between nearby Android and desktop devices.

Compared with other collaborative approaches, membership management is simple and requires minimal configuration. However, these relationships are usually temporary and do not persist beyond individual file transfers.

3.5.2 Synchronization Model

Local sharing technologies are not designed to maintain synchronized copies of files across multiple devices. Instead, they focus on transferring files from one device to another as individual sharing operations.

For example, after a file has been transferred using *AirDrop*, subsequent modifications are not propagated automatically. Additional transfers must be initiated manually whenever updated versions need to be shared.

As a result, these systems provide direct and efficient file exchange but do not support continuous synchronization or multi-device replication.

3.5.3 History Management

Local sharing technologies generally provide little or no support for history management. Files are transferred as standalone artifacts, and previous versions are not tracked by the sharing mechanism itself.

For example, neither *AirDrop* nor *Nearby Share* maintains revision histories, rollback mechanisms, or conflict resolution facilities. Any version management must therefore be performed by external tools or by users manually.

3.5.4 File Support

Local sharing technologies typically support a wide range of file types, including documents, source code, images, multimedia content, datasets, and other binary assets.

For example, *AirDrop* can transfer photos, videos, documents, and application-specific files directly between devices. Similar support is provided by *Nearby Share*.

Because files are transferred directly rather than processed by a synchronization service, heterogeneous file types are generally handled without special restrictions.

3.5.5 Filesystem Transparency and Usability

Local sharing technologies are designed for simplicity and ease of use. File transfers are usually initiated through familiar graphical interfaces and require little technical knowledge.

For example, users can share files through operating-system integration in *AirDrop* and *Nearby Share* without interacting with specialized collaboration workflows or repository management tools.

This approach provides strong usability and low setup effort, making local sharing technologies particularly suitable for quick exchanges between nearby devices.

3.6 Summary of Trade-offs

The comparison presented in the previous subsections highlights that existing approaches emphasize different aspects of collaborative file management. Centralized collaboration platforms provide strong support for user management, synchronization, and usability. Collaborative version control systems offer comprehensive history management and conflict resolution capabilities. Peer-to-peer synchronization systems focus on decentralized synchronization and transparent filesystem integration, while local sharing technologies prioritize simplicity and direct device-to-device file transfer.

To provide a consolidated view of these trade-offs, Tables 2, 3, 4, 5, and 6 summarize the relative support provided by each approach across the evaluation dimensions introduced in Section 3.1. The qualitative ratings used in the comparison are defined in Table 1.

Table 1: Qualitative Rating Scale

Rating	Meaning
High	Strong support for the capability
Partial	Partial support with notable limitations
Limited	Basic or restricted support
None	Capability not provided

Table 2: Comparison of Existing Approaches: Membership Management

Category	System	Peer Discovery	Access Control	Group Formation
Centralized	*Google Drive*	Limited	High	High
	OneDrive	Limited	High	High
	Dropbox	Limited	High	High
	Nextcloud	Limited	High	Partial
Version Control	*Git*	None	None	Limited
	SVN	None	High	Limited
	Perforce	None	High	Limited
P2P Sync	*Syncthing*	High	Partial	Partial
	Resilio Sync	Partial	Partial	Partial
Local Sharing	*AirDrop*	High	None	Limited
	Nearby Share	High	None	Limited

Table 3: Comparison of Existing Approaches: Synchronization Model

Category	**System**	**Automatic Synchronization**	**Offline Operation**	**Multi-Device Synchronization**
Centralized	*Google Drive*	High	Limited	High
	OneDrive	High	Limited	High
	Dropbox	High	Limited	High
	Nextcloud	High	Partial	High
Version Control	*Git*	None	High	Partial
	SVN	None	Limited	Partial
	Perforce	None	Limited	Partial
P2P Sync	*Syncthing*	High	High	High
	Resilio Sync	High	High	High
Local Sharing	*AirDrop*	None	None	None
	Nearby Share	None	None	None

Table 4: Comparison of Existing Approaches: History Management

Category	**System**	**Version History**	**Rollback**	**Conflict Resolution**
Centralized	*Google Drive*	Limited	Limited	Limited
	OneDrive	Limited	Limited	Limited
	Dropbox	Limited	Limited	Limited
	Nextcloud	Partial	Partial	Limited
Version Control	*Git*	High	High	High
	SVN	High	High	High
	Perforce	High	High	High
P2P Sync	*Syncthing*	Partial	Partial	Limited
	Resilio Sync	Partial	Partial	Limited
Local Sharing	*AirDrop*	None	None	None
	Nearby Share	None	None	None

Table 5: Comparison of Existing Approaches: File Support

Category	**System**	**Heterogeneous Files**	**Large Files**
Centralized	*Google Drive*	High	Partial
	OneDrive	High	Partial
	Dropbox	High	Partial
	Nextcloud	High	High
Version Control	*Git*	Partial	Limited
	SVN	Partial	Partial
	Perforce	High	High
P2P Sync	*Syncthing*	High	High
	Resilio Sync	High	High
Local Sharing	*AirDrop*	High	High
	Nearby Share	High	High

Table 6: Comparison of Existing Approaches: Filesystem Transparency and Usability

Category	**System**	**Filesystem Integration**	**Ease of Use**
Centralized	*Google Drive*	High	High
	OneDrive	High	High
	Dropbox	High	High
	Nextcloud	High	Partial
Version Control	*Git*	Limited	Limited
	SVN	Limited	Limited
	Perforce	Limited	Limited
P2P Sync	*Syncthing*	High	High
	Resilio Sync	High	High
Local Sharing	*AirDrop*	High	High
	Nearby Share	High	High

4 Identified Gap

4.1 Limitations of Centralized Solutions

Centralized collaboration platforms provide strong support for membership management, synchronization, file compatibility, and usability. However, these advantages are achieved through reliance on centralized infrastructure for storage, coordination, and access management.

As a result, collaboration depends on the availability of a central service and its associated network connectivity. Although these platforms support basic version recovery, they generally provide limited facilities for long-term revision tracking and conflict management when compared with dedicated version control systems.

Furthermore, ownership and control of shared data remain tied to the centralized platform. This model may be unsuitable for temporary collaborative groups seeking lightweight and serverless file sharing.

4.2 Limitations of Collaborative Version Control Systems

Collaborative version control systems provide strong support for revision tracking, rollback, and conflict resolution. These capabilities make them highly effective for managing the evolution of shared project artifacts over time.

However, such systems are primarily designed for controlled revision management rather than transparent file synchronization. Changes are typically exchanged through explicit operations such as commits, pushes, pulls, or updates, requiring users to actively participate in the synchronization process.

In addition, version control workflows often introduce a higher level of complexity than ordinary file sharing solutions. Repository management, branching, merging, and conflict resolution require specialized tools and technical knowledge that may be unnecessary for small temporary collaboration groups.

Furthermore, support for large binary files and multimedia assets may be limited depending on the system being used. As a result, collaborative version control systems are well suited for managing project history but are less suitable as lightweight directory synchronization solutions.

4.3 Limitations of Existing Peer-to-Peer Solutions

Existing peer-to-peer approaches address different aspects of collaborative file management, but none provide comprehensive support across all evaluation dimensions.

Collaborative version control systems such as *Git*, *SVN*, and *Perforce* offer strong history management through version tracking, rollback, and conflict resolution. However, they rely on explicit workflows and are not designed for transparent directory synchronization.

Conversely, peer-to-peer synchronization systems such as *Syncthing* and *Resilio Sync* provide automatic directory synchronization and strong filesystem integration. Nevertheless, their history management capabilities are typically limited to basic version retention and recovery mechanisms.

Local sharing technologies such as *AirDrop* and *Nearby Share* further simplify file exchange, but they do not provide persistent synchronization, collaborative history management, or long-term shared workspaces.

4.4 Limitations of Local Sharing Technologies

Local sharing technologies simplify file exchange by enabling direct device-to-device transfers between nearby users. Systems such as *AirDrop* and *Nearby Share* provide convenient mechanisms for sharing files without requiring centralized infrastructure or complex setup.

However, these technologies are designed primarily for one-time transfers rather than persistent collaboration. They do not maintain synchronized directories, automatically propagate subsequent modifications, or preserve revision history.

In addition, sharing relationships are typically temporary and must be re-established whenever files are exchanged. As a result, local sharing technologies are highly effective for quick file transfers but do not provide the functionality required for ongoing collaborative file management.

4.5 The Gap

The limitations discussed in the previous sections show that existing approaches provide different strengths for collaborative file management, but none fully satisfies the combination of requirements.

Table 7 summarizes the performance of the reviewed approaches across the five evaluation dimensions introduced in Chapter 3: membership management, synchronization, history management, file support, and filesystem transparency and usability. The comparison shows that each category focuses on a particular subset of these dimensions.

Centralized collaboration platforms provide strong support for membership management, synchronization, file support, and usability, but offer only limited history management. Collaborative version control systems provide comprehensive history management, but provide weaker support for synchronization, file support, and transparent filesystem integration. Peer-to-peer synchronization systems provide strong synchronization and usability, but only limited support for membership management and version recovery. Local sharing technologies simplify file exchange and provide strong usability, but do not support persistent synchronization or history management.

Consequently, the gap identified in this study is not the absence of any individual capability. Rather, the gap is the absence of a lightweight solution that simultaneously provides strong support across all five evaluation dimensions. Existing systems typically optimize one aspect of collaborative file management at the expense of others.

Therefore, there remains a need for a lightweight peer-to-peer directory synchronization system that combines decentralized collaboration, automatic synchronization, basic version recovery, support for heterogeneous project files, and transparent filesystem integration within a single solution.

Local Sharing Technologies

5 Positioning Your Project

5.1 Key Differentiators

TODO (team): add key differentiators.

5.2 Design Philosophy

TODO (team): add design philosophy.

Table 7: Gap Summary Across Evaluation Dimensions

Approach	Membership Management	Synchronization	History Management	File Support	Filesystem Transparency & Usability
Centralized Platforms	High	High	Limited	High	High
Version Control Systems	Partial	Limited	High	Partial	Limited
P2P Synchronization Systems	Partial	High	Limited	High	High
	Limited	None	None	High	High
Desired Solution	High	High	High	High	High

6 Research to Application Mapping

To translate academic theories into our proposed architecture, we map eight core research insights to planned components in our Go/C++ design:

1. **Zero-Configuration Peer Discovery on LAN**
   - **Research Insight:** Cheshire and Krochmal [7,8] establish that Multicast DNS (mDNS) and DNS-Based Service Discovery (DNS-SD) enable serverless local service registration and resolution using standard IP multicast queries.
   - **Application in Project:** We plan a Go-based peer discovery component that broadcasts service records under the `_p2psync._tcp` domain and dynamically browses the local subnet to resolve peer addresses, removing the need for manual configuration.
2. **Causal Ordering of Concurrent Updates**
   - **Research Insight:** Lamport [9] demonstrates that logical clocks and causal relation tracking can order events in a distributed system without relying on synchronized physical clocks.
   - **Application in Project:** We plan to attach per-file logical timestamps (Lamport clocks) to track change sequences and detect concurrent modifications, avoiding the state and coordination overhead of vector clocks for the initial release.
3. **Consistency Strategies under Network Partitions**
   - **Research Insight:** Gilbert and Lynch's proof of the CAP Theorem [10] establishes that distributed systems must trade consistency for availability during network partitions.
   - **Application in Project:** We choose availability by allowing peers to make local modifications offline. When connection is restored, a resync process runs to resolve conflicts and converge on a common state.
4. **Conflict Handling and Durability**
   - **Research Insight:** Replicated storage systems like Bayou [11] demonstrate that resolving conflicts requires deterministic policies paired with non-destructive version backups to ensure data durability.

- **Application in Project:** We plan to use a Last-Write-Wins (LWW) policy based on logical timestamps. Before overwriting a local file, we plan to store a backup copy to protect against data loss.

5. **Conflict-Free Convergence Evolution**
   - **Research Insight:** Shapiro et al. [12] prove that Conflict-Free Replicated Data Types (CRDTs) allow concurrent updates to merge automatically and converge mathematically.
   - **Application in Project:** We plan to use LWW for the initial release to reduce complexity, but construct our SQLite metadata schema and Go coordination logic to support state-based CRDTs in future versions.
6. **Bandwidth-Efficient Synchronization**
   - **Research Insight:** Tridgell and Mackerras [13] show that rolling checksums allow remote directories to synchronize by transferring only modified blocks instead of entire files.
   - **Application in Project:** We plan to start with whole-file transfers for simplicity. In a later optimization phase, we will add block-level diffing to minimize local WLAN bandwidth use.
7. **Kernel-Level File Watching**
   - **Research Insight:** Operating system event APIs like Linux's inotify [14] and macOS's FSEvents [15] provide kernel-level event streams for file additions, modifications, and deletions without the overhead of periodic polling.
   - **Application in Project:** We plan a C++ file watcher daemon that uses native OS APIs (inotify and FSEvents) to detect directory events with minimal CPU overhead, notifying the Go coordination engine via IPC.
8. **Data Integrity and Reliable Transport**
   - **Research Insight:** Cryptographic hashing standard SHA-256 [16] provides unique content identifiers that verify data integrity, while the Transmission Control Protocol (TCP) [17] guarantees reliable, ordered packet delivery.
   - **Application in Project:** We plan to let the C++ daemon hash local files for change verification, and let the Go network coordinator transfer files over TCP sockets to ensure error-free replication.

7 Key References

The following six publications serve as the primary theoretical and technical foundation for our proposed system's architecture and design choices.

1. **RFC 6762 & RFC 6763 (Stuart Cheshire and Marc Krochmal, 2013)** [7, 8]
   These specifications define Multicast DNS (mDNS) and DNS-Based Service Discovery (DNS-SD), which form the basis for zero-configuration networking. They demonstrate how local devices can advertise and resolve services without a centralized DNS server. We use these protocols to plan our Go-based peer discovery component, allowing team members on the same local network to connect automatically and sync directories without manual setup.

2. **Leslie Lamport (1978) - "Time, Clocks, and the Ordering of Events in a Distributed System"** [9]
   This paper introduces logical clocks to track event ordering and causality in distributed systems where physical clocks cannot be synchronized. It provides the foundation for our synchronization logic. We plan to attach a single logical timestamp counter to each file metadata entry. This enables our Go coordinator to detect concurrent modifications and resolve update sequencing chronologically without the complexity of full vector clocks.

3. **Seth Gilbert and Nancy Lynch (2002) - "Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services"** [10]
   This work provides the formal proof of the CAP Theorem, showing that a distributed system cannot guarantee consistency, availability, and partition tolerance simultaneously. This theorem guides our architectural choice to prioritize availability. We allow peers to edit files locally while offline and reconcile changes eventually upon reconnection, which fits the dynamic connection cycles of student teams.

4. **Marc Shapiro et al. (2011) - "A Comprehensive Study of Convergent and Commutative Replicated Data Types"** [12]
   This research report analyzes CRDTs, demonstrating how data structures can achieve eventual consistency mathematically without consensus protocols. While we adopt a simpler Last-Write-Wins policy for our initial release to reduce complexity, this study serves as our design roadmap, ensuring our SQLite schema and Go metadata models can support state-based CRDTs in future iterations.

5. **Andrew Tridgell and Paul Mackerras (1996) - "The Rsync Algorithm"** [13]
   This paper presents a rolling-checksum algorithm that synchronizes directories by transferring only modified file blocks. It forms the basis of our planned bandwidth optimization. In our design, we begin with whole-file transfer pipelines, and plan to integrate this block-level delta sync in a later phase to optimize local WLAN usage for large binary assets.

6. **Karin Petersen et al. (1995) - "Managing Update Conflicts in Bayou, a Weakly Connected Replicated Storage System"** [11]
   This paper explores conflict management and reconciliation in weakly connected replicated storage systems. It introduces conflict detection and resolution policies, informing our decision to combine an automated Last-Write-Wins (LWW) resolution rule with local backup preservation. This ensures predictable reconciliation while keeping older file versions accessible to prevent accidental data loss.

8 Conclusion

8.1 Literature Review Synthesis

Our analysis of distributed systems literature highlights that building a serverless, local directory sync tool requires combining data-integrity algorithms with network and clock synchronization standards. mDNS and DNS-SD [7, 8] remove the need for central registry servers, while logical clocks [9] establish event ordering in decentralized networks. Because partitions are expected on local subnets, the CAP Theorem [10] suggests prioritizing availability and eventual convergence. Deterministic resolution policies combined with version backups [11] provide a safe conflict-handling baseline, with CRDTs [12] offering a path toward automatic conflict-free convergence.

8.2 Influence on Design

The literature shapes our proposed Go/C++ architecture:

- We plan to use mDNS/DNS-SD (via Go) to automate local network peer discovery, removing setup friction.
- We plan to use logical timestamps (Lamport clocks) and a Last-Write-Wins conflict resolution policy to manage concurrent edits predictably.
- We plan to favor local availability, ensuring users can work offline and sync differences upon reconnection.
- We plan a split where Go handles network coordination and C++ handles OS-level file watching (inotify [14], FSEvents [15]) and hashing (SHA-256 [16]) over reliable TCP sockets [17].

8.3 Limitations and Future Work

While our current design provides a synchronization baseline, we recognize limitations in bandwidth usage and conflict handling. Whole-file transfers are simple to implement but inefficient for large assets. Our next steps are:

- Implement the Go-to-C++ socket handover pipeline to connect the network and storage layers.
- Implement the rsync-style rolling checksum algorithm [13] in C++ to enable delta transfers.
- Introduce state-based CRDT schemas [12] in Go to support automatic, conflict-free metadata merging.

References

[1] Google, "Google drive help: Use drive for desktop," https://support.google.com/drive/, 2026, accessed: 2026-05-28.

[2] Dropbox, "Dropbox help center," https://help.dropbox.com/, 2026, accessed: 2026-05-28.

[3] S. Chacon and B. Straub, *Pro Git*, 2nd ed. Apress, 2014.

[4] GitHub, "Github docs: About repositories," https://docs.github.com/en/repositories/creating-and-managing-repositories/about-repositories, 2026, accessed: 2026-05-28.

[5] S. Community, "Syncthing documentation," https://docs.syncthing.net/, 2026, accessed: 2026-05-28.

[6] I. Resilio, "Resilio sync documentation," https://help.resilio.com/hc/en-us, 2026, accessed: 2026-05-28.

[7] S. Cheshire and M. Krochmal, "Multicast dns," RFC 6762, 2013, accessed: 2026-05-28. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6762

[8] ——, "Dns-based service discovery," RFC 6763, 2013, accessed: 2026-05-28. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6763

[9] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978.

[10] E. A. Brewer, "Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services," *ACM SIGACT News*, vol. 33, no. 2, pp. 51–59, 2002.

[11] K. Petersen, M. Spreitzer, D. Terry, M. Theimer, and A. Demers, "Managing update conflicts in bayou, a weakly connected replicated storage system," in *Proceedings of the 15th ACM Symposium on Operating Systems Principles*, 1995, pp. 172–182.

[12] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, "A comprehensive study of convergent and commutative replicated data types," *INRIA Research Report*, no. RR-7506, 2011.

[13] A. Tridgell and P. Mackerras, "The rsync algorithm," in *Proceedings of the USENIX Annual Technical Conference*, 1996.

[14] M. Kerrisk, "inotify(7) — linux manual page," https://man7.org/linux/man-pages/man7/inotify.7.html, 2024, accessed: 2026-05-28.

[15] Apple, "File system events programming guide," https://developer.apple.com/library/archive/documentation/Darwin/Conceptual/FSEvents_ProgGuide/Introduction/Introduction.html, 2007, accessed: 2026-05-28.

[16] N. I. of Standards and Technology, "Fips pub 180-4: Secure hash standard (shs)," 2015, accessed: 2026-05-28. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf

[17] W. M. Eddy, "Transmission control protocol (tcp)," RFC 9293, 2022, accessed: 2026-05-28. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9293